CASE STUDY

# Fraud Detection & Business Impact Analytics

How a 3.5% fraud rate translates into a $430K decision framework — and why the right threshold depends on your business, not your model

---

Rajneesh Chauhan | B.Tech Information Technology, Lucknow Institute of Technology (2026)

Dataset: IEEE-CIS Fraud Detection (Kaggle) | 590,540 Transactions | June 2026

github.com/Codex610/fraud-detection-business-impact-analytics

# Introduction

This case study documents the end-to-end analysis of credit card fraud detection using the IEEE-CIS Fraud Detection dataset. It follows the data analysis process: **Ask, Prepare, Process, Analyze, Share, and Act**. The project goes beyond measuring model accuracy — it frames every analytical decision as a business question and translates model outputs into quantified financial impact.

> **Core question this project answers:** How do you build a fraud detection system that a business can actually use — not just one that scores well on a leaderboard? And what is the right decision threshold when "optimal" depends entirely on how much a false positive actually costs your company?

## Scenario

You are a junior data analyst at a financial analytics firm. Your client — a payment processing company — is losing money to fraudulent transactions and wants a smarter detection system than their current manual review process, which catches only about 15% of fraud. They need a model that flags suspicious transactions, a clear picture of what it costs when the model gets it wrong, and a framework for deciding how aggressively to flag transactions based on their risk tolerance.

## Case Study Roadmap

| Phase | Key Question | Deliverable |
|---|---|---|
| Ask | What is the business problem? | Business task statement |
| Prepare | What data do we need and where is it? | Data source description |
| Process | How was the data cleaned? | Cleaning documentation |
| Analyze | What did the data reveal? | Analysis summary + findings |
| Share | How do we communicate findings? | Visualizations + key insights |
| Act | What should the business do next? | Recommendations + limitations |

PHASE 1

# Ask — Defining the Business Problem

## Business Task Statement

Build a transaction-level fraud detection pipeline that: (1) identifies fraudulent transactions with measurable precision and recall, (2) translates those predictions into a net dollar-savings framework, and (3) determines the optimal decision threshold not by statistical convention but by actual business cost assumptions.

## Five Guiding Questions

**1. What type of company is the client and what do they need?**

A payment processor losing revenue to fraud. They need a model that flags suspicious transactions before they are settled, with a clear cost-benefit framework for how aggressively to flag.

**2. What are the key factors involved?**

Class imbalance (only 3.5% of transactions are fraud), the cost of false positives (legitimate customers wrongly blocked), the cost of false negatives (fraud that slips through), and the decision threshold that balances these two costs.

**3. What type of data is appropriate?**

Transaction-level data with behavioral, identity, and card attributes. Time of transaction, product category, card network, and email domain are all relevant signals.

**4. Where does the data come from?**

IEEE-CIS Fraud Detection dataset (Kaggle). 590,540 real transactions with 434 features across two tables: transaction details and identity information.

**5. Who is the audience and how will findings be presented?**

Two audiences: (a) fraud operations teams who need to know which transactions to review, and (b) finance leadership who need to know the dollar impact of each threshold choice. Different outputs serve each.

## Key Stakeholders

| Stakeholder | What They Care About | Metric That Matters |
|---|---|---|
| Fraud Operations Team | Which transactions to review | Recall, False Negatives |
| Finance Leadership | Dollar cost of fraud and of blocking legit customers | Net Savings, FP Cost |
| Product / Risk Teams | Customer experience impact | False Positive Rate |
| Data / Analytics Team | Model reliability and explainability | AUC-PR, SHAP |

PHASE 2

# Prepare — Data Sources and Integrity

## Data Source

The IEEE-CIS Fraud Detection dataset was sourced from Kaggle. It was created by Vesta Corporation, a real fraud prevention service provider, making it one of the most realistic publicly available fraud datasets available.

| Attribute | Detail |
|---|---|
| Source | IEEE-CIS Fraud Detection — Kaggle Competition |
| Provider | Vesta Corporation (real payment processing company) |
| License | Kaggle competition data — for research and educational use |
| Total Records | 590,540 transactions |
| Tables | 2 — train_transaction.csv + train_identity.csv |
| Raw Features | 434 columns across both tables |
| Target Variable | isFraud (binary: 0 = legitimate, 1 = fraud) |
| Fraud Rate | 3.50% (20,663 fraud / 590,540 total) |
| Total Amount | $79,738,948 |
| Fraud Amount | $3,083,845 (3.87% of total amount) |
| Time Span | Not disclosed — TransactionDT is seconds from a reference point |

## Data Credibility Assessment (ROCCC)

| Criterion | Assessment | Notes |
|---|---|---|
| Reliable | Yes | Sourced from a real fraud prevention company |
| Original | Yes | Direct from Vesta Corporation via Kaggle |
| Comprehensive | Partial | V-columns and some C-columns are anonymized |
| Current | Partial | Exact dates not disclosed; patterns may differ from today |
| Cited | Yes | IEEE-CIS 2019 competition — publicly documented |

## Known Limitations of This Dataset

- Vesta's V* columns (157 features) are fully anonymized — their real meaning is unknown, which limits explainability
- C* columns represent counts (addresses, devices, emails per card) but exact definitions are not documented
- The exact time range of transactions is not disclosed — annualized projections require unverifiable assumptions
- The dataset was collected from a specific industry context; fraud patterns vary by merchant type and geography

PHASE 3

# Process — Cleaning and Preprocessing

## Tools Used

Python 3.11 with pandas, NumPy, scikit-learn, XGBoost, SHAP, matplotlib, and seaborn. All work was done in Jupyter Notebooks with outputs saved to a structured reports/ folder.

## Cleaning Steps — Documented

### Step 1: Table Merge

Merged train_transaction and train_identity on TransactionID using a left join. Left join preserves all 590,540 transactions; identity data is absent for ~76% of rows, which is itself a fraud signal (fraudsters often have no identity link).

### Step 2: Drop High-Missing Columns

208 columns with more than 70% missing values were dropped. These were primarily identity columns (id_21 through id_27) with 99%+ missing rates. No imputation strategy recovers a 99%-empty column.

### Step 3: Impute Numeric Columns

Count-based columns (C1-C14) were filled with 0 — absence of a count means zero occurrences, not unknown. Time-delta columns (D*) and Vesta behavioral columns (V*) were filled with the column median, which is robust to the outliers fraud data produces.

### Step 4: Impute Categorical Columns

Missing categorical values (email domain, card network) were filled with "unknown" rather than the mode. This is deliberate — a missing email domain is itself a fraud signal. Filling with the mode would hide that signal.

### Step 5: Encode Categoricals

Low-cardinality categoricals (10 or fewer unique values) were label encoded. High-cardinality columns (e.g., P_emaildomain) were frequency encoded — each category replaced by its occurrence rate. One-hot encoding was avoided because it would have produced 200+ sparse columns.

### Step 6: Remove Low-Variance Columns

25 columns where one value appeared in 99%+ of rows were dropped. These carry near-zero predictive signal.

## Preprocessing Summary

| Metric | Before | After |
|---|---|---|
| Total Rows | 590,540 | 590,540 |
| Total Columns | 436 | 204 |

| | | |
|---|---|---|
| Columns Dropped (missing) | 208 | — |
| Columns Dropped (low variance) | 25 | — |
| Missing Values | 414 columns affected | 0 (zero) |
| Categorical Encoding | 13 object columns | All numeric |

PHASE 4

# Analyze — Findings from the Data

## EDA Finding 1: Fraud is Rare but High-Value

Only 3.50% of transactions are fraudulent, but they account for 3.87% of total transaction value — meaning fraudulent transactions are slightly higher in average dollar value than legitimate ones. This confirms that a model predicting everything as "not fraud" achieves 96.5% accuracy while being completely useless. Accuracy is the wrong metric here.

## EDA Finding 2: Product Category is the Strongest Risk Signal


*Figure 1: ProductCD "C" has an 11.69% fraud rate — 3x the dataset average of 3.5%. Category W has the highest transaction volume but the lowest fraud rate (2.04%).*

ProductCD "C" has a fraud rate of 11.69% — more than triple the dataset average. Category "W" dominates in volume (440K transactions) but has the lowest fraud rate (2.04%). This distinction matters operationally: a blanket fraud rule applied across all products would flag too many W transactions and miss the real risk in C.

## EDA Finding 3: Fraud Peaks at 6-9 AM, Not Late Night

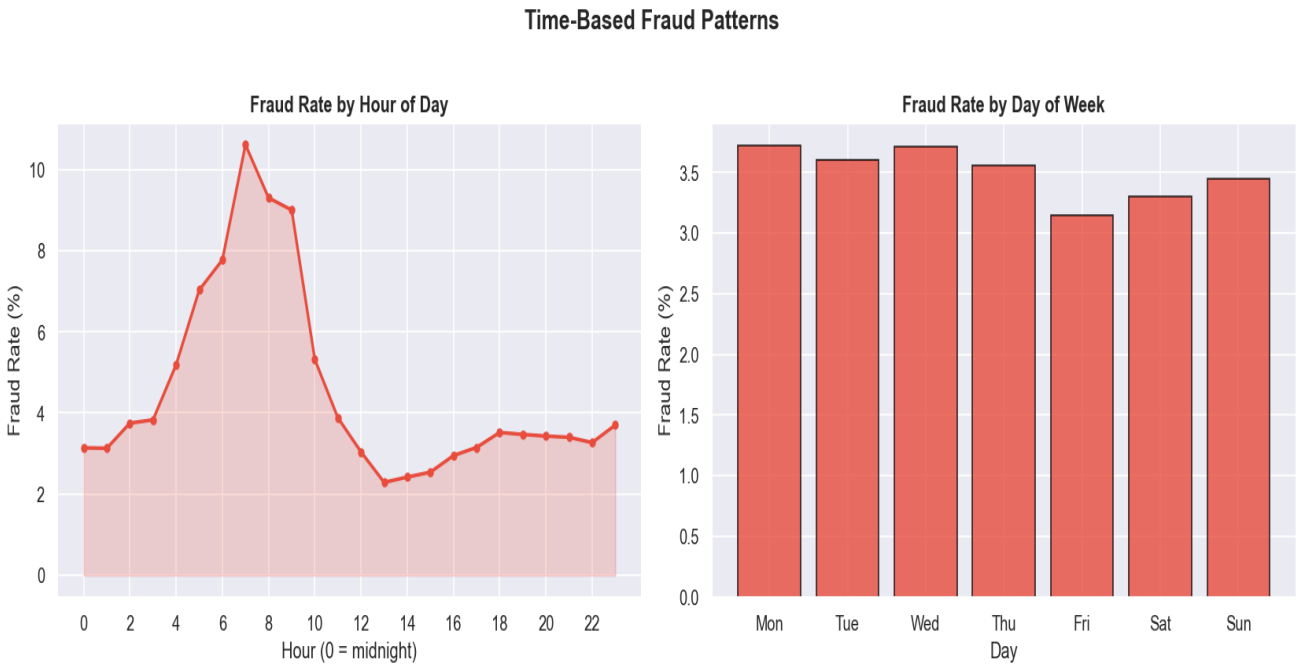


*Figure 2: Fraud rate spikes between 6 AM and 9 AM (peaking at 10.5%). Day of week shows minimal variation — time of day is the stronger signal.*

The common assumption that fraud happens at night is wrong in this dataset. Fraud peaks sharply between 6 AM and 9 AM at over 10% fraud rate — roughly 3x the baseline. Day of week has near-zero predictive value (3.1% to 3.7% across all days). This means time-of-day features should be engineered, but day-of-week features add noise rather than signal.

## Feature Engineering — What Was Built and What Actually Worked

22 features were engineered across five categories. SHAP analysis revealed that only 3 of the 22 added measurable predictive value above what raw columns already captured. This is reported honestly — building features does not guarantee they will be useful.

| Feature Group | Features Built | SHAP Rank Range | Verdict |
|---|---|---|---|
| Card Velocity | card1_txn_count, card1_amt_mean, amt_deviation | Rank 6-8 | Useful |
| Risk Scores | product_card_risk, card4/card6 risk scores | Rank 1, 59-61 | Mixed |
| Time Features | is_peak_fraud_hour, time_of_day, is_weekend | Rank 52-179 | Weak |
| Amount Features | amt_bucket, is_high_value, is_round_amount | Rank 33-171 | Weak |
| V/C Aggregates | v_cols_sum/std, c_cols_sum/max/mean | Rank 34-104 | Weak |

**Key insight from feature engineering:** The individual C1, C14, and C13 columns outperformed the aggregated c_cols_sum by 5-10x in SHAP importance. Aggregating them destroyed the granularity that made them individually predictive. Aggregation is a convenience, not a guaranteed improvement — always validate with SHAP.

## Modeling — Baseline vs XGBoost

A Logistic Regression baseline was established before training XGBoost. This discipline matters: if a complex model does not substantially outperform a simple one, the complexity is not justified. XGBoost used scale_pos_weight to handle the 27.6:1 class imbalance natively — no synthetic oversampling (SMOTE) was needed.

| Model | Train AUC-ROC | Test AUC-ROC | AUC-PR | Notes |
|---|---|---|---|---|
| Logistic Regression | 0.8524 | 0.8477 | — | Class weight: balanced |
| XGBoost | 0.9580 | 0.9399 | 0.6350 | scale_pos_weight = 27.6 |
| Improvement | +10.56pp | +9.22pp | — | Train-test gap: 0.018 (no overfitting) |

**Why AUC-PR matters more than AUC-ROC here:** With 96.5% legitimate transactions, the model can score 0.84 AUC-ROC simply by being good at the easy majority class. AUC-PR = 0.635 is the honest number — it measures performance specifically on the rare fraud class where the business problem actually lives.

## Critical Finding: Target Leakage Was Caught and Fixed

In the initial version of the pipeline, product_card_risk, card4_risk_score, and card6_risk_score were computed using the full dataset before the train/test split. This leaked test-set fraud labels into training features — a common and often undetected error that inflates reported performance. The fix: recompute all risk scores using only training data (smoothed target encoding), then apply the mapping to the test set. Unseen test categories fall back to the global training fraud rate.

PHASE 5

# Share — Visualizations and Key Findings

## Finding 1: AUC-PR is the Honest Metric

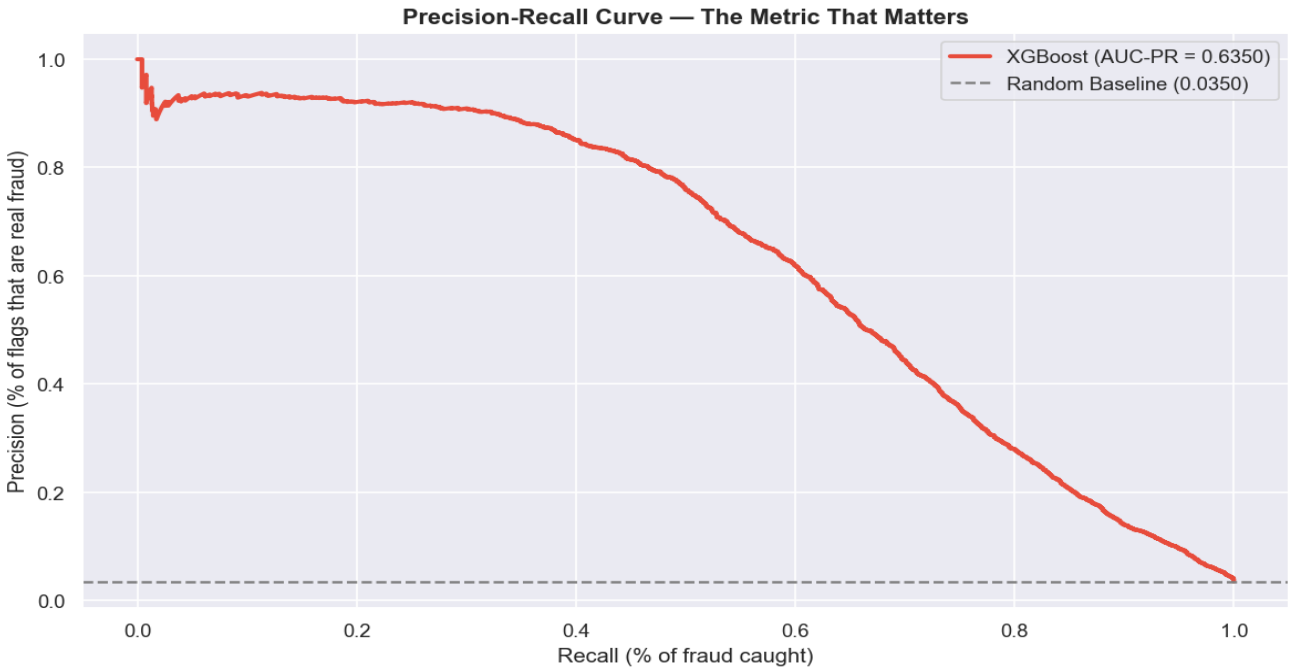


*Figure 3: Precision-Recall curve for XGBoost (AUC-PR = 0.6350) vs random baseline (0.0350). The gap between these lines represents the model's real predictive value on the fraud class.*

The precision-recall curve shows that at high precision thresholds (above 0.90), recall drops sharply — the model catches very few fraud cases when it is being extremely conservative. The business implication: there is no free lunch. Catching more fraud always means accepting more false positives.

## Finding 2: Confusion Matrix at Threshold 0.80

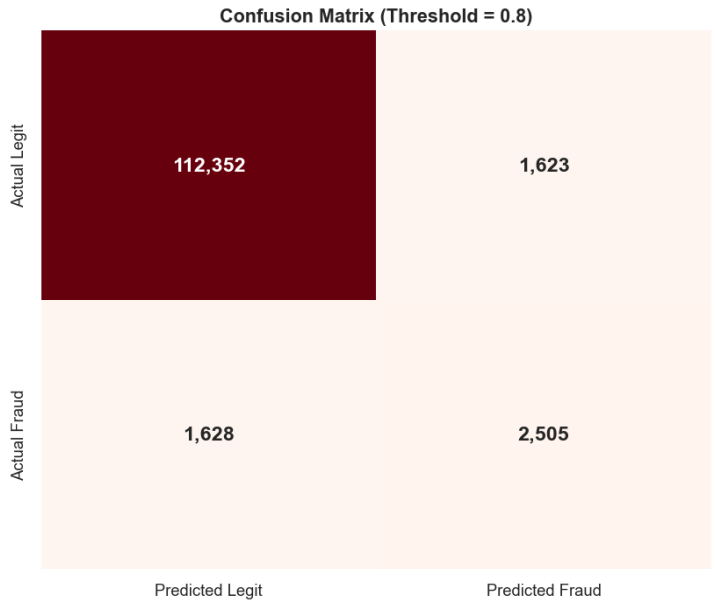


*Figure 4: Confusion matrix at threshold = 0.80 (F1-optimal). 2,505 fraud cases correctly caught. 1,628 fraud cases missed. 1,623 legitimate transactions wrongly flagged.*

| Outcome | Count | Business Meaning |
|---|---|---|
| True Negatives | 112,352 | Legitimate transactions correctly passed — no cost |
| False Positives | 1,623 | Real customers wrongly blocked — friction + churn risk |
| False Negatives | 1,628 | Fraud that slipped through — direct financial loss |
| True Positives | 2,505 | Fraud correctly caught — money saved |

## Finding 3: SHAP Explains What the Model Actually Uses

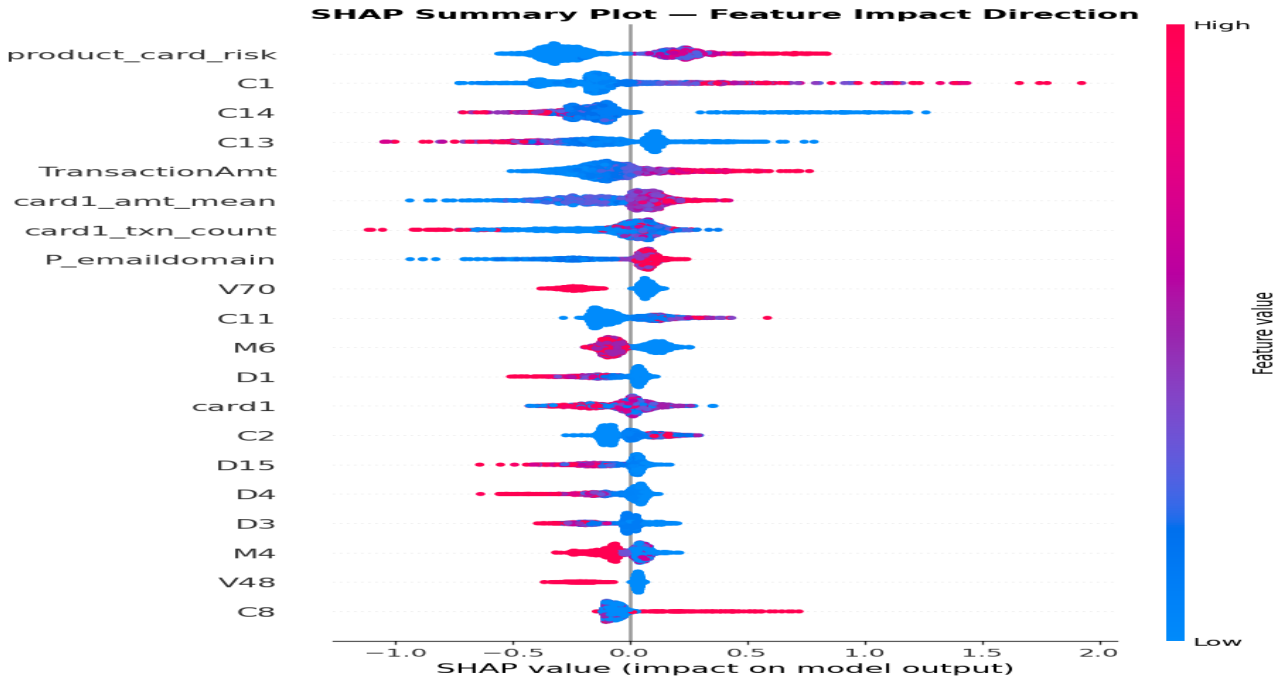


*Figure 5: SHAP summary plot. Red = pushes prediction toward fraud. Blue = pushes toward legitimate. Features sorted by mean absolute impact. product_card_risk (engineered feature) ranks #1 — validating the risk-encoding logic.*

Three observations from the SHAP plot that a simple feature importance chart would miss:

- product_card_risk is the top feature — confirming that combining product category and card type into a single risk score captured signal that neither variable had alone.
- C1, C14, C13 (Vesta count features) rank 2nd, 3rd, 4th — meaning the number of linked addresses/emails/devices per card is a strong fraud predictor, even though we do not know the exact definitions.
- Time-based features (is_peak_fraud_hour, is_weekend) ranked in the bottom 30 despite the clear 6-9 AM pattern visible in EDA. The V/C columns already captured that signal indirectly — making explicit time flags redundant.

## Finding 4: The Threshold Decision is a Business Call, Not a Model Output

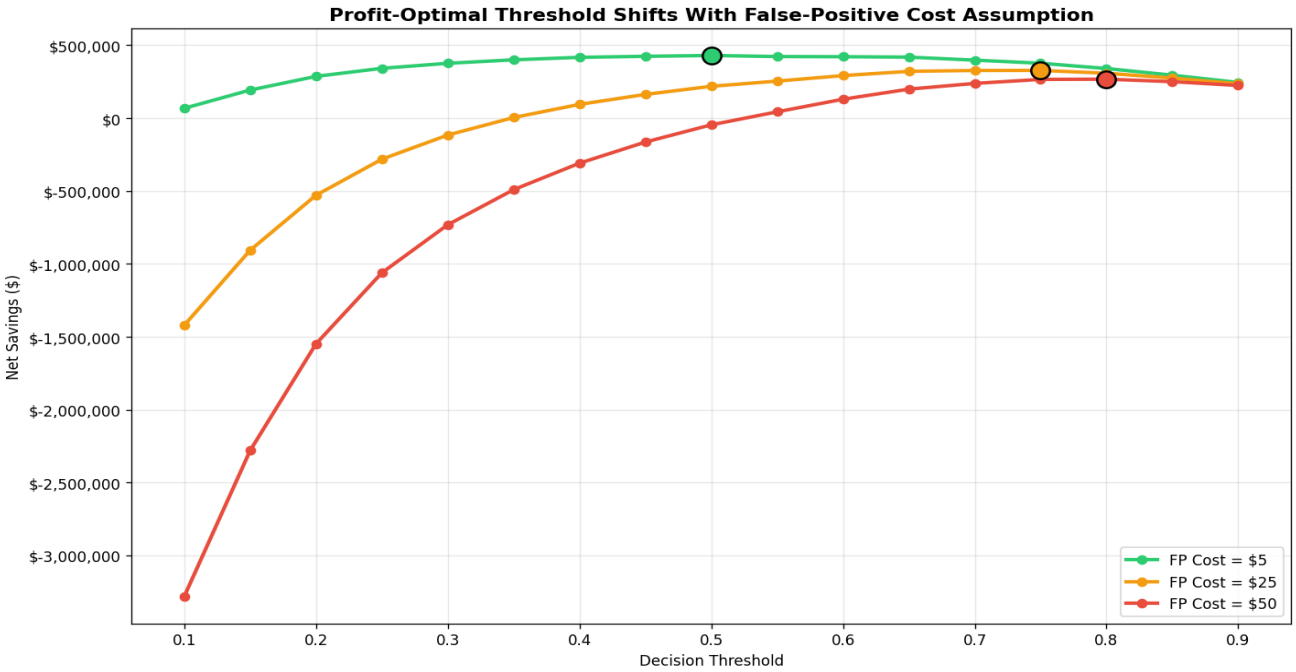


*Figure 6: Net savings across thresholds under three false-positive cost assumptions. Optimal threshold shifts from 0.50 (low FP cost) to 0.80 (high FP cost). The F1-optimal threshold (0.80) leaves $89K unrealized savings when FP cost is low.*

This is the most important finding in the project. The F1-optimal threshold (0.80) is not the profit-optimal threshold. Under a $5 false-positive cost assumption, using threshold 0.50 instead of 0.80 recovers an additional $89,512 in net savings on the test set. The right threshold depends on what the company's false-positive cost actually is — and that is a business data question, not a modeling question.

## Cost-Sensitivity Summary

| FP Cost Assumption | Optimal Threshold | Net Savings | Fraud Caught | False Positives |
|---|---|---|---|---|
| $5 (low friction) | 0.50 | $430,064 | 3,414 | 10,574 |
| $25 (medium) | 0.75 | $327,581 | 2,704 | 2,463 |
| $50 (high churn risk) | 0.80 | $267,517 | 2,505 | 1,623 |

**Important caveat:** The $5, $25, and $50 false-positive cost figures are illustrative assumptions — not measured values. Real FP cost requires data on customer churn rate after a blocked legitimate transaction, average customer lifetime value, and support escalation cost. This framework is designed so that plugging in real numbers immediately produces the correct threshold recommendation.

PHASE 6

# Act — Recommendations and Next Steps

## Top High-Level Insights

**1. AUC-ROC is the wrong success metric for fraud detection.**
A model predicting everything as legitimate gets 96.5% accuracy. AUC-PR (0.635) is the correct primary metric — it reflects performance specifically on the rare fraud class where the business problem lives.

**2. ProductCD "C" deserves separate treatment.**
At 11.69% fraud rate, Category C transactions are more than 3x riskier than the dataset average. A risk operations team should apply stricter review rules to this category specifically, not treat all products equally.

**3. The fraud peak at 6-9 AM is operationally actionable.**
Staffing fraud review teams to be at full capacity during this window — rather than during overnight hours — would improve real-time catch rates.

**4. F1 score and profit are optimized at different thresholds.**
The F1-optimal threshold (0.80) is not the profit-optimal threshold (0.50 at low FP cost). Deploying the model with the F1 threshold leaves $89K in unrealized savings in this test set alone. Threshold should be set by finance/risk, not by data science.

**5. Target leakage is a silent, common error.**
Risk-encoded features computed before the train/test split inflated apparent performance. Any model going to production should be audited for this class of error — it is not caught by standard evaluation metrics.

## Recommended Business Actions

- Deploy the model at a threshold chosen by the risk/finance team using the cost-sensitivity framework, not the F1-optimal default
- Collect real false-positive cost data (churn rate after blocked transactions, support escalation rate) to replace the illustrative $5/$25/$50 assumptions
- Apply stricter manual review rules to ProductCD "C" transactions regardless of model score — the base rate alone justifies heightened scrutiny
- Staff fraud review teams at full capacity between 6 AM and 9 AM, when fraud rate is 3x the daily average
- Re-run SHAP analysis quarterly — fraud patterns shift over time and feature importance will drift

## Additional Analysis for Further Exploration

- Full interaction analysis for C1 and C14 — both show non-linear, interaction-heavy SHAP patterns that a single dependence plot does not fully explain
- Network analysis of P_emaildomain — building a graph of email-to-card relationships may surface organized fraud rings not visible in single-transaction analysis

- Temporal drift analysis — are the same fraud patterns present in the first half of the dataset versus the second half? Model decay detection is critical for production deployment
- Merchant-level risk scoring — if merchant ID data were available, merchant-level fraud rates would likely be a stronger feature than product category alone
- Customer lifetime value modelling — to convert the illustrative FP cost assumptions into measured values derived from actual churn data

## Stated Limitations

- Vesta's V* and C* features are anonymized — top SHAP features cannot be given real-world interpretations
- False-positive cost assumptions ($5/$25/$50) are illustrative — not derived from company data
- The dataset's time span is unknown — any annualized savings projection is an extrapolation, not a measured result
- The model was trained on 2019 competition data — fraud patterns have likely shifted since then

---

| Metric | Result |
|---|---|
| Dataset | 590,540 transactions — IEEE-CIS Fraud Detection |
| Fraud Rate | 3.50% (severe class imbalance) |
| Baseline AUC-ROC | 0.8477 (Logistic Regression) |
| XGBoost AUC-ROC | 0.9399 |
| XGBoost AUC-PR | 0.6350 (honest metric for this problem) |
| F1-Optimal Threshold | 0.80 — Precision 60.7%, Recall 60.6% |
| Net Savings at F1 Threshold | $340,552 |
| Net Savings at Profit Threshold | $430,064 (threshold 0.50, FP cost = $5) |
| Gap | $89,512 left unrealized by using F1 threshold |
| Critical Error Found | Target leakage in risk-encoded features — corrected |
| SHAP Finding | 3 of 22 engineered features added real predictive value |

*Project repository: github.com/Codex610/fraud-detection-business-impact-analytics*

*Author: Rajneesh Chauhan — B.Tech Information Technology, Lucknow Institute of Technology (2026)*